

Absolutely. Based on your original ideas and the current PSA/Tuas ecosystem, I would frame the proposal around **augmenting PSA's existing automation rather than replacing it**. PSA already has highly automated cranes, AGVs, AI-supported yard planning and an expanding digital/supply-chain ecosystem; the gap your team can target is **cross-system disruption orchestration**. ([PSA International](#))

Below is a proposal you can adapt directly for your hackathon submission.

PORTWATCH

AI-Powered Disruption Orchestration for Tuas Port

Tagline

When disruptions happen, Portwatch knows what to do — and when to ask a human.

1. Executive Summary

Tuas Port is being developed as a highly automated and digitally integrated port ecosystem. PSA already employs automated quay cranes, Automated Guided Vehicles (AGVs), automated yard cranes and AI-based planning technologies to improve throughput and operational efficiency. Its wider Tuas ecosystem is also increasingly connected through digital supply-chain solutions. ([PSA International](#))

However, operational disruptions do not occur within a single system.

A vessel arriving late can create berth conflicts, which can affect crane allocation, AGV demand, yard utilisation and truck appointments. Similarly, an equipment fault can trigger a chain of downstream operational consequences. During such incidents, operators may need to gather information from multiple systems, evaluate competing recovery options and determine which actions can safely be automated and which require human approval.

Portwatch is an AI-powered disruption orchestration agent designed to address this gap.

Rather than replacing PSA's existing systems, Portwatch acts as an intelligent coordination layer across them. It receives operational alerts, state changes, process metrics and user requests; analyses their downstream impact; generates recovery options; executes low-risk actions automatically; and escalates high-impact or uncertain decisions to human operators.

Every action is accompanied by a structured execution trace containing the input, analysis, tool calls, decisions, confidence levels, approvals, actions, results and errors.

The core principle is:

Automate the response where it is safe. Escalate where it is not.

2. Problem Statement

The challenge

PSA has made substantial investments in automation and digitalisation at Tuas Port.

The terminal already incorporates technologies such as automated quay cranes, AGVs, automated yard cranes and AI-supported planning. PSA's recent developments also include digital solutions designed to coordinate container and supply-chain flows. ([PSA International](#))

The challenge is therefore not simply:

"How do we automate port operations?"

Instead, it is:

"How do we coordinate increasingly automated systems when an unexpected disruption crosses operational boundaries?"

Consider a single incident:

Vessel ETA changes

- berth schedule changes
- crane assignments need to be reconsidered
- AGV demand changes
- containers may be redirected
- yard capacity becomes constrained
- truck appointments may need to change
- another vessel may be affected.

A human operator must potentially reason across all of these dependencies before deciding what to do.

This creates three key challenges:

1. Fragmented operational decisions

Different systems provide information about different parts of the operation. A disruption may require information from several systems before a safe decision can be made.

2. Cascading consequences

A seemingly small disruption can propagate across the terminal.

3. Unequal risk

Not every decision should receive the same level of autonomy.

Changing a notification is very different from changing a vessel schedule that could breach an SLA or moving a dangerous-goods container.

3. Why Existing Automation Is Not Enough

Portwatch is not designed to compete with PSA's existing automation.

In fact, **existing automation is the foundation of the solution.**

PSA's Tuas Port already uses automated equipment and intelligent planning systems. PSA describes Tuas as a highly automated terminal using AGVs, automated quay/yard equipment and AI-based planning. ([PSA International](#))

The opportunity is to add a layer above these systems:

Existing systems answer:

"What is happening in my system?"

Portwatch asks:

"What does this event mean for the entire operation, what should happen next, and am I allowed to do it autonomously?"

This makes Portwatch an **orchestration layer**, not another standalone optimisation system.

4. Proposed Solution

Portwatch

Portwatch continuously monitors operational signals and creates a unified view of disruptions.

Inputs

Portwatch can receive:

- Vessel ETA changes
- Crane breakdown alerts
- AGV availability changes
- Yard congestion metrics
- Gate queue metrics
- Weather events
- Dangerous-goods exceptions
- Equipment health alerts
- Operator/user requests

These inputs are normalised into a common incident representation.

5. Core Agent Workflow

Portwatch follows an eight-stage loop:

1. Detect

Identify an operational event or user request.

2. Correlate

Determine whether multiple events are related to the same incident.

3. Analyse

Determine which vessels, berths, cranes, AGVs, yard blocks and downstream processes may be affected.

4. Plan

Generate multiple possible recovery actions.

5. Simulate

Estimate the consequences of each option before execution.

6. Apply policy

Determine whether the proposed action can be executed autonomously.

7. Execute or escalate

Execute safe actions or request human approval.

8. Verify and trace

Confirm the outcome and record the entire decision process.

6. Main Innovation: Controlled Autonomy

The most important feature of Portwatch is that it does not treat maximum autonomy as the goal.

Instead, Portwatch uses **tiered autonomy**.

Tier 1 — Auto Execute

Low-risk, reversible actions.

Examples:

- Send operational notifications
- Refresh a schedule
- Re-route a low-priority job
- Adjust a non-critical appointment
- Update an internal status

Action: Execute automatically.

Tier 2 — Execute + Notify

Moderate-impact actions that fall within predefined operational limits.

Examples:

- Reassign an available backup crane
- Adjust AGV task priorities
- Modify a limited number of truck appointments

Action: Execute automatically and notify the relevant operator.

Tier 3 — Human Approval

High-impact, irreversible, safety-sensitive or uncertain actions.

Examples:

- Potential contractual SLA breach
- High-priority vessel affected
- Significant financial impact
- Safety-critical equipment decision
- Dangerous-goods handling
- Low-confidence decision caused by missing or unreliable data

Action: Stop and request human approval.

7. Why the Policy Engine Is Separate From the AI

A central design principle is that the LLM should **not be the final authority on whether it is allowed to execute an action**.

The AI may propose:

"Reassign Crane #7."

The deterministic policy engine then evaluates:

- Is the crane available?
- Is the action reversible?
- What is the predicted delay?
- Does it breach an SLA?
- What is the cost?
- Is dangerous goods involved?
- Is confidence above the required threshold?
- Does the action require human approval?

Only after passing this gate can an action be executed.

This provides a safety boundary between:

AI reasoning

and

operational authority.

8. Flagship Demo Scenario

"The Three-Way Disruption"

The demo follows one continuous incident at Tuas Port.

Initial state

Vessel:

MSC Anna

Berth:

B3

Scheduled arrival:

10:00

Scheduled departure:

18:00

Yard destination:

Block C7

Event 1 — Vessel delay

At 09:00:

MSC Anna ETA changes from 10:00 to 11:30.

Portwatch detects the state change.

It checks:

- Current berth allocation
 - Other vessel schedules
 - Crane assignments
 - Yard capacity
-

Event 2 — Crane failure

At 09:05:

Crane #4 reports a hydraulic fault.

Portwatch correlates this with the vessel delay rather than treating it as an independent alert.

It determines:

MSC Anna → Berth B3 → Crane #4 → 62 remaining container moves

Event 3 — Yard congestion

The yard system reports:

Block C7 utilisation = 91%

Portwatch identifies another constraint.

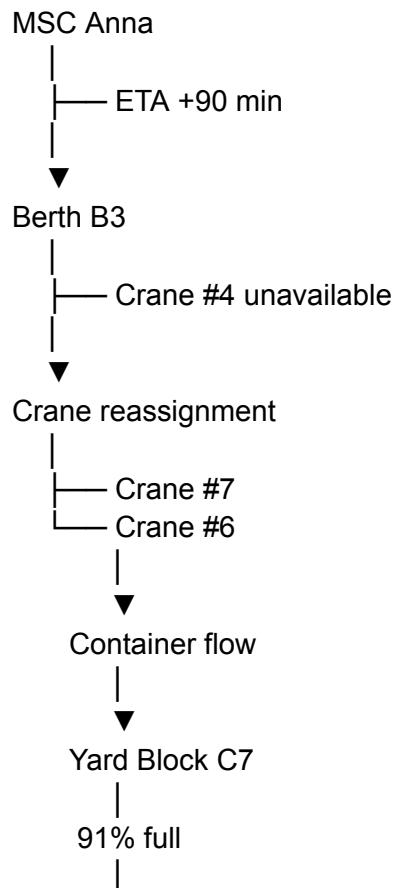
Simply redirecting additional containers to C7 may worsen congestion.

The agent now has to consider:

- Alternative cranes
 - Alternative yard blocks
 - AGV availability
 - Vessel departure time
 - Yard capacity
 - Downstream truck impact
-

9. Impact Analysis

Portwatch constructs an operational impact graph:





Truck / AGV demand

This lets the operator see **why a disruption matters**, rather than simply receiving another alert.

10. Recovery Planning

Portwatch generates three possible recovery plans.

Option A — Crane #7 only

Predicted vessel delay:

+2 hours

Cost:

Low

Yard impact:

Medium

Risk:

Low

Option B — Crane #6 + Crane #7

Predicted vessel delay:

+40 minutes

Cost:

Medium

Yard impact:

Medium

Risk:

Medium

Option C — Change berth sequence

Predicted delay:

+20 minutes

Cost:

High

Customer impact:

High

Risk:

Medium

Portwatch recommends:

Option B — Reassign Crane #6 and Crane #7.

However, it does not immediately execute the plan.

It first evaluates the action through the policy engine.

11. Demonstrating Tool Failure

During the demo, the Crane #7 telemetry service deliberately times out.

Portwatch records:

Crane #7 telemetry request

↓

TIMEOUT

↓

Retry
↓
TIMEOUT
↓
Use last-known state
↓
Confidence reduced

The system explicitly tells the operator:

**Crane #7 telemetry unavailable. Last-known state is 2 minutes old.
Confidence reduced from 91% to 67%.**

Portwatch does not silently invent missing information.

This demonstrates the ability to handle:

- incomplete information
 - unreliable tools
 - API failures
 - uncertainty
 - confidence degradation.
-

12. Human Escalation

The simulation predicts:

MSC Anna departure: 18:40

Contractual target:

18:00

Therefore:

Potential SLA breach detected.

The policy engine classifies the action as **Tier 3**.

Portwatch stops autonomous execution.

The operator sees:

HUMAN APPROVAL REQUIRED

MSC Anna — Berth B3

Situation

Crane #4 has failed.

Crane #7 telemetry is temporarily unavailable.

Yard Block C7 is at 91% utilisation.

Recommended option

Option B

Reassign Crane #6 + Crane #7.

Predicted impact

- Departure delay: +40 min
- Yard utilisation: 89%
- Cost: Medium
- Confidence: 67%
- SLA: At risk

Alternatives

Option A: Lower cost, larger delay

Option C: Higher cost, lower vessel delay

Decision

[Approve] [Reject] [Modify]

13. Human-in-the-Loop

The Ops Duty Manager approves Option B.

Portwatch then executes:

1. Update crane assignments
2. Update vessel schedule
3. Update yard allocation
4. Adjust AGV priorities
5. Notify affected teams
6. Update downstream transport information

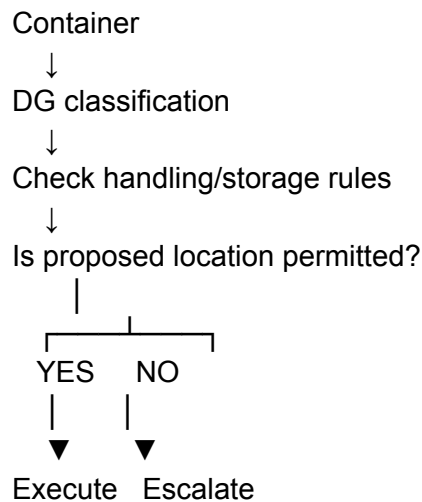
Each action is verified individually.

If one tool fails, Portwatch records the failure instead of claiming success.

14. Dangerous Goods Safety Layer

Portwatch can also incorporate dangerous-goods constraints.

For example, if a container affected by the recovery plan is identified as dangerous goods:



This ensures that throughput optimisation cannot override safety constraints.

The solution would use the applicable DG rules and operational data rather than allowing the LLM to infer safety requirements. Singapore's dangerous-goods regime incorporates the IMDG framework and requires appropriate declaration/documentation for dangerous goods. ([MPA](#))

15. User Request Capability

During the incident, a planner can ask:

"What is the current status of MSC Anna?"

Portwatch retrieves the live incident state and execution trace.

Example:

MSC Anna — CURRENT STATUS

Berth:

B3

Incident:

Crane #4 hydraulic failure

Recovery:

Crane #6 + #7 reassigned

Yard:

Block C7 — 89%

Current predicted departure:

18:40

SLA:

At risk

Human approval:

Approved

Recovery:

In progress

The answer is grounded in the system's operational state rather than generated from generic model knowledge.

16. Execution Trace

Every incident produces a structured audit trail.

09:00:01

VESSEL ETA CHANGE DETECTED

09:00:02

MSC Anna ETA:

10:00 → 11:30

09:00:03

TOS queried

✓ Berth B3 confirmed

09:05:11

CRANE #4 FAULT DETECTED

09:05:12

Crane Scheduler queried

✓ Crane #7 available

✓ Crane #6 available

09:05:13

Yard Manager queried

⚠ Block C7 = 91%

09:05:15

Recovery plans generated

09:05:17

Crane #7 telemetry requested

09:05:22

⚠ TIMEOUT

09:05:23

Retry initiated

09:05:28

⚠ RETRY FAILED

09:05:29

Last-known telemetry used

09:05:29

Confidence:
91% → 67%

09:05:31
Policy Engine evaluated Option B

09:05:31
● HUMAN APPROVAL REQUIRED

09:07:02
Ops Duty Manager approved Option B

09:07:03
✓ Crane Scheduler updated

09:07:04
✓ TOS updated

09:07:05
✓ Yard allocation updated

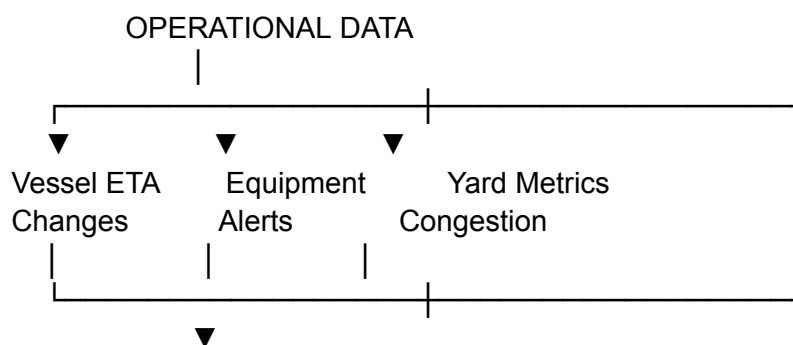
09:07:06
✓ Notifications sent

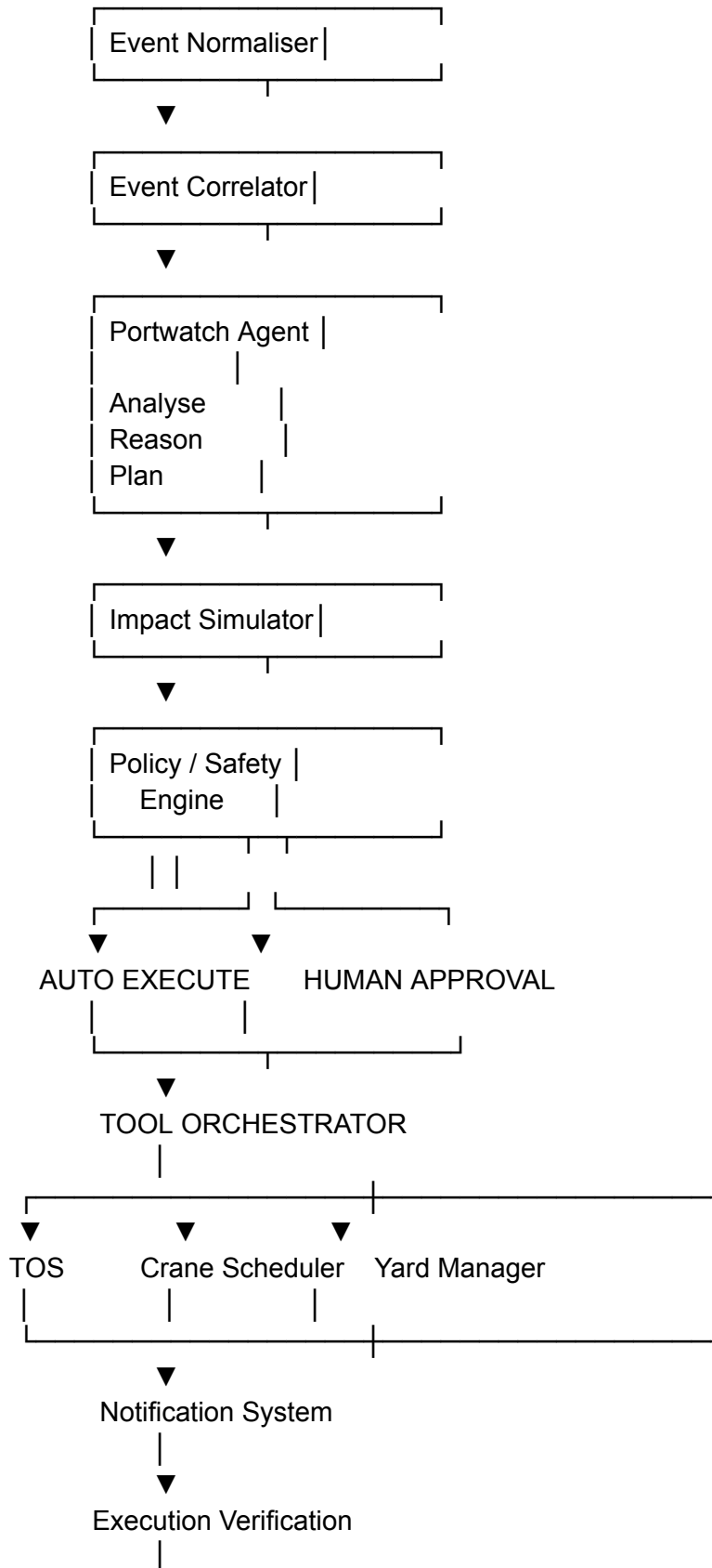
09:07:08
✓ Recovery verified

The execution trace is not merely a debugging log.

It is an **auditability and trust mechanism**.

17. System Architecture





18. Technology Stack

Frontend

React / Next.js

Dashboard containing:

- Live incident feed
 - Port map
 - Impact graph
 - Agent activity
 - Recovery options
 - Human approval panel
 - Execution trace
-

Backend

Python + FastAPI

Responsible for:

- Event ingestion
 - Agent orchestration
 - Tool APIs
 - Incident management
 - Policy engine
 - Execution trace
-

AI Layer

An LLM with tool-calling capabilities.

The model handles:

- Natural-language interpretation
- Incident analysis
- Planning
- Tool selection
- Recovery-option generation
- Explanation

The LLM does **not** directly control safety-critical execution.

Policy Engine

Deterministic Python rules.

Example:

```
IF
    action.reversible == TRUE
AND
    predicted_sla_breach == FALSE
AND
    safety_risk == LOW
AND
    confidence >= 0.80
```

```
THEN
    AUTO_EXECUTE
```

Otherwise:

```
REQUIRE_HUMAN_APPROVAL
```

Simulated PSA Systems

For the hackathon, actual PSA APIs are unnecessary.

We can create mock services:

```
/TOS
/crane-scheduler
/yard-manager
/agv-manager
```

/gate-manager
/dg-checker
/notification

Each returns realistic operational JSON.

19. Why a Simulation Is Appropriate

The objective of the prototype is not to reproduce PSA's proprietary systems.

Instead, the prototype demonstrates the **agent architecture**.

Mock APIs allow the team to demonstrate:

- tool orchestration
- system integration
- failures
- retries
- uncertainty
- policy decisions
- human approval
- execution verification

The same interfaces could theoretically be connected to real operational systems in a future deployment.

20. Security and Safety

Least privilege

The agent should only have access to the tools required for each action.

For example:

Notification Agent
→ notification permission

Scheduling Agent

→ schedule permission

Yard Agent

→ yard allocation permission

No single agent component receives unrestricted access.

Policy enforcement outside the LLM

Safety rules cannot be overridden through model reasoning.

The LLM can recommend an action, but the policy engine determines whether that action is permitted.

Human override

Operators can:

- Reject an action
 - Modify a recommendation
 - Cancel execution
 - Override an automated decision
-

Kill switch

A global emergency control can disable autonomous execution.

Auditability

Every important decision is logged:

- Input
- Timestamp
- Tool call
- Tool response

- Model decision
 - Confidence
 - Policy result
 - Human approval
 - Execution result
 - Error
-

21. Handling Uncertainty

Portwatch follows a simple principle:

Uncertainty should reduce autonomy, not disappear from the interface.

For example:

High confidence

91%



Auto-execution possible

Medium confidence

67%



Additional verification / restricted actions

Low confidence

42%



Human approval required

This prevents the agent from treating incomplete information as certainty.

22. Scalability

Portwatch is designed around an event-driven architecture.

Multiple incidents can be processed independently:

Incident #101

Crane failure



Portwatch Worker

Incident #102

Yard congestion



Portwatch Worker

Incident #103

Vessel ETA change



Portwatch Worker

The same architecture could eventually support:

- Berth disruptions
- Yard congestion
- Equipment failures
- Gate congestion
- DG exceptions
- Reefer incidents
- Predictive maintenance

The agent becomes a reusable **operational decision layer** rather than a single-purpose application.

23. Expected Impact

Portwatch aims to improve four areas.

1. Faster incident response

Instead of operators manually gathering information from multiple systems, Portwatch immediately correlates relevant signals.

2. Reduced cascading disruption

By evaluating downstream effects, the agent can intervene before a local disruption becomes a terminal-wide problem.

3. Better human decision-making

Operators receive:

- Situation summary
- Impact analysis
- Recovery options
- Predicted consequences
- Confidence
- Recommended action

rather than raw alerts.

4. Safer automation

Low-risk decisions can happen automatically while high-risk decisions remain under human control.

24. Success Metrics

For the hackathon prototype, we can measure:

Response time

Time from:

disruption detected → **recommended action**

Decision quality

Compare the agent's selected recovery plan against predefined optimal/simulated outcomes.

Autonomous resolution rate

Percentage of incidents that can safely be resolved without human intervention.

Escalation accuracy

Percentage of high-risk incidents correctly escalated.

SLA protection

Reduction in predicted vessel/SLA delays compared with the baseline plan.

Tool-failure resilience

Percentage of simulated tool failures where the agent:

- retries correctly
 - falls back safely
 - reduces confidence
 - escalates appropriately.
-

25. Hackathon MVP

We deliberately keep the first version focused.

Must have

Input

Three event types:

1. Vessel ETA change
2. Crane failure
3. Yard congestion

Agent

- Analyse incident
- Correlate events
- Query tools
- Generate recovery plans

Policy

Three autonomy tiers:

- Auto
- Auto + notify
- Human approval

Tools

Mock:

- TOS
- Crane scheduler
- Yard manager
- Notification service

Failure simulation

At least one API timeout.

Human approval

Approve / reject recovery plan.

Execution trace

Timeline of the entire incident.

26. Future Extensions

After the MVP, Portwatch could expand to:

Gate congestion

Predict truck queues and adjust appointments.

Predictive maintenance

Use equipment telemetry to recommend:

- Monitor
- Schedule maintenance
- Remove equipment from service

Reefer monitoring

Detect temperature excursions and coordinate technician response.

Dangerous goods

Automatically identify safety constraints before proposing yard movements.

Weather response

Predict weather-related operational impact and adjust schedules.

Supply-chain orchestration

Extend beyond the terminal into the wider Tuas ecosystem.

PSA's planned Supply Chain Hub @ Tuas is intended to strengthen integration between the port and broader logistics ecosystem, making this type of cross-system orchestration increasingly relevant. The facility was announced as scheduled for completion in 2027. ([PSA International](#))

27. Why Portwatch Is Different

Portwatch is not:

Another dashboard

because it does not just display information.

It reasons about what should happen.

Portwatch is not:

Another chatbot

because it can call tools and execute operational workflows.

Portwatch is not:

Another optimisation algorithm

because it incorporates human approval, uncertainty and operational policies.

Portwatch is:

An agentic operational control layer that connects signals, reasoning, tools and human decisions.

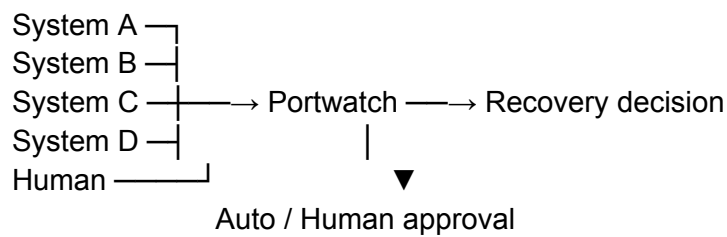
28. Innovation

The key innovation is **controlled cross-system autonomy**.

Traditional automation:

System A
optimises A

Portwatch:



The system does not seek maximum automation.

It seeks:

The highest safe level of autonomy for each decision.

29. Alignment With PSA's Direction

Portwatch builds upon PSA's existing technology direction rather than competing against it.

PSA has positioned Tuas Port as a highly automated terminal and a wider ecosystem connecting port, logistics and supply-chain activities. ([PSA International](#))

PSA's 2025 reporting also highlights the continuing digitalisation of the Tuas ecosystem, including iWX and other solutions aimed at improving container and supply-chain flows. ([PSA Annual Report](#))

Therefore, Portwatch can be positioned as:

The intelligent coordination layer that helps existing automated systems respond to unexpected situations collectively.

30. Three-Minute Demo Script

0:00 — The problem

Show the Tuas operational dashboard.

"Tuas Port is increasingly automated. But when disruption happens, the problem is no longer automating individual machines. The challenge is coordinating the entire operation."

0:20 — Disruption

Trigger:

MSC Anna ETA delayed by 90 minutes.

Then:

Crane #4 hydraulic failure.

Then:

Yard Block C7 reaches 91% utilisation.

0:40 — Agent investigates

Portwatch automatically queries:

- TOS
- Crane scheduler
- Yard manager
- AGV manager

The impact graph appears.

"The agent has identified that this isn't three separate alerts. They are part of the same operational disruption."

1:10 — Recovery planning

Portwatch generates:

Option A

Option B

Option C

Then:

"Option B provides the best recovery outcome."

1:30 — Failure

Simulate Crane #7 telemetry failure.

Show:

TIMEOUT

→ RETRY

→ FAILURE

→ CACHED DATA

→ CONFIDENCE 67%

1:50 — Human escalation

The simulation predicts an SLA breach.

Policy engine says:

HUMAN APPROVAL REQUIRED

Show the approval screen.

2:10 — Human decision

Operator clicks:

APPROVE OPTION B

Portwatch executes the plan.

2:30 — Verification

Show:

TOS ✓

Crane Scheduler ✓

Yard ✓

Notifications ✓

2:40 — User query

Planner asks:

"What's the current status of MSC Anna?"

Portwatch answers using the live incident state.

2:50 — Execution trace

Show the complete timeline.

3:00 — Closing

"Portwatch doesn't replace the people or automation already running the port. It connects them when disruption happens — acting autonomously when it is safe, and knowing when a human needs to be in control."

31. Final Pitch

PORTWATCH

From alerts to action — safely.

PSA has already built one of the world's most advanced automated port environments.

The next challenge is not simply **more automation**.

It is **better coordination when things go wrong**.

A delayed vessel, failed crane or congested yard block can trigger a cascade of consequences across the terminal.

Portwatch acts as an AI disruption-response layer across these systems.

It:

Detects → Correlates → Analyses → Simulates → Decides → Executes → Verifies → Escalates

Low-risk actions happen automatically.

High-impact decisions go to humans.

And every decision leaves a complete execution trail.

Portwatch: AI that doesn't just know what to do — it knows when it should not act alone.